Algorithms for Massive Data

Experimental Project

Done by: Chaima Ben Fredj
ID: 56374A

Project 2:
Market-Basket Analysis

IMDB Top 1000 Movies Dataset

University of Milan
Data Science for Economics
A.Y. 2025/2026

Contents

1 Dataset

The project uses the **IMDB Top 1000 Movies and TV Shows** dataset, published on Kaggle under the CC0 Public Domain license by Harshit Shankhdhar. The dataset contains 1000 rows, each representing one of the highest-rated movies or TV shows on IMDB, with 16 columns including title, genre, release year, IMDB rating, and the four leading actors (Star1, Star2, Star3, Star4).

The dataset is downloaded programmatically during code execution using the Kaggle API:

```
os.environ['KAGGLE_USERNAME'] = "xxxxxx"
os.environ['KAGGLE_KEY']      = "xxxxxx"
!kaggle datasets download -d harshitshankhdhar/imdb-dataset-of-top
   -1000-movies-and-tv-shows
```

Only the four actor columns (Star1–Star4) are used in this project. All other columns are ignored.

2 Data Organisation

The market-basket model is applied as follows:

- **Baskets:** each movie is treated as one basket
- **Items:** the four billed actors (Star1, Star2, Star3, Star4)

Each movie is converted into a `frozenset` of its four actor names, giving a collection of 1000 baskets. A `frozenset` is used because it is unordered (actors have no ranking within a basket) and hashable (required for use as dictionary keys in the algorithms).

The dataset statistics after loading are:

Statistic	Value
Total baskets (movies)	1000
Distinct actors	2709
Average basket size	4.00

For the A-Priori algorithm, the baskets are additionally converted into a boolean matrix using `TransactionEncoder` from the `mlxtend` library. This matrix has 1000 rows (movies) and 2709 columns (actors), where each cell is `True` if the actor appears in that movie and `False` otherwise.

3 Pre-processing

The pre-processing steps applied to the data are minimal, as the dataset is already well-structured:

1. **Column selection:** only the four actor columns (Star1–Star4) are retained.
2. **Missing value handling:** `pd.notna()` is used to skip any empty actor cells when building baskets, though no missing values were found in the actor fields.

3. **Support threshold:** a global support threshold of $s = 5$ is used, corresponding to 0.5% of the 1000 movies. This value was chosen to be low enough to capture meaningful co-occurrence patterns while high enough to exclude coincidental appearances.

4 Algorithms and Implementations

Three algorithms from the course are implemented and compared: A-Priori, PCY, and SON.

4.1 A-Priori

The A-Priori algorithm exploits the **monotonicity property**: if an itemset is frequent, all its subsets must also be frequent. It proceeds in passes:

- **Pass 1:** count all singleton actors $\rightarrow$ find L_1 (frequent actors)
- **Pass 2:** count only pairs of actors both in $L_1 \rightarrow$ find L_2
- **Pass k:** count k-itemsets whose $(k-1)$-subsets are all in L_{k-1}

This avoids counting itemsets that cannot possibly be frequent, which is the main advantage over a naive approach.

In this project, A-Priori is implemented using the `mlxtend` library:

```
frequent_itemsets = mlxtend_apriori(data_encoded,
                        min_support=s/len(baskets),
                        use_colnames=True)
```

The `min_support` parameter takes a fraction rather than an absolute count, so the threshold is passed as $s/1000 = 0.005$.

4.2 PCY (Park–Chen–Yu)

PCY improves on A-Priori by using the spare memory available on Pass 1 to maintain a hash table of pair counts. The algorithm runs as follows:

Pass 1: in addition to counting individual actors (same as A-Priori), every pair of actors in each basket is hashed to a bucket in a hash table of size 1009 (chosen as a prime number for better distribution), and that bucket's count is incremented.

At the end of Pass 1, the bucket counts are converted to a **bitmap**: bucket b gets value 1 if its count $\geq s$ (frequent bucket), and 0 otherwise. A bucket with count $< s$ means that no pair hashing to it can be frequent.

Pass 2: a pair $\{i, j\}$ is counted only if:

1. both i and j are in L_1 (same condition as A-Priori), **and**
2. the pair hashes to a frequent bucket (bitmap value = 1)

This second condition is the key distinction from A-Priori and reduces the number of pairs that need to be counted.

In the experiment, Pass 1 identified 689 out of 1009 buckets as frequent (68.3%), meaning 31.7% of buckets were used to filter out candidate pairs before Pass 2.

Passes 3 and beyond use the same candidate generation logic as A-Priori.

4.3 SON (Savasere, Omiecinski & Navathe)

SON avoids false negatives through the following guarantee: *if an itemset is frequent globally, it must be frequent in at least one chunk.* The algorithm runs in two passes:

Pass 1: the baskets are split into $c = 5$ equal chunks of 200 movies each. A-Priori is run on each chunk with a reduced local support threshold:

$$s_{\text{local}} = \max\left(2,\ \text{round}\left(s \cdot \frac{|\text{chunk}|}{n}\right)\right) = \max(2, \text{round}(5 \cdot 0.2)) = 2$$

All locally frequent itemsets from all chunks are collected into a set of **candidates**. After Pass 1, 378 candidates were identified across the 5 chunks.

Pass 2: every candidate is counted on the full dataset of 1000 baskets. Only candidates with global count $\geq s = 5$ are kept as truly frequent.

This structure maps naturally to MapReduce: in Pass 1, each chunk can be processed in parallel by a separate Map task, and in Pass 2 each candidate can be counted in parallel across subsets of the data.

5 Scalability

To evaluate how the algorithms scale with data size, all three are run on 20%, 40%, 60%, 80%, and 100% of the dataset. The support threshold is scaled proportionally: $s_{\text{sub}} = \max(2,\ \text{round}(s \cdot \text{frac}))$.

Fraction	Baskets	A-Priori (s)	PCY (s)	SON (s)
20%	200	0.01413	0.00272	0.02225
40%	400	0.08153	0.00797	0.03182
60%	600	0.04272	0.00700	0.05689
80%	800	0.04051	0.00964	0.10593
100%	1000	0.03282	0.01135	0.17409

Table 1: Runtime (seconds) of each algorithm at different dataset sizes.

PCY is consistently the fastest algorithm at every dataset size. SON shows a steeper growth in runtime: from 0.022s at 200 baskets to 0.174s at 1000 baskets, due to the overhead of running 5 separate A-Priori passes on chunks plus a full global counting pass. This overhead does not pay off at 1000 movies but would become an advantage at much larger scales where data cannot fit in memory and parallel processing of chunks is necessary.

6 Experiments

All three algorithms are run on the full dataset of 1000 movies with support threshold $s = 5$.

6.1 Frequent Itemsets

Size	Count	Description
L_1	79	Frequent individual actors
L_2	3	Frequent actor pairs
L_3	1	Frequent actor triples

Table 2: Number of frequent itemsets by size ($s = 5$).

The top 10 most frequent individual actors are:

Actor	Movies
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Matt Damon	11
Christian Bale	11
Leonardo DiCaprio	11
James Stewart	10
Denzel Washington	9

Table 3: Top 10 most frequent actors in the IMDB Top 1000 dataset.

The three frequent pairs and the one frequent triple are all from the Harry Potter cast:

Pair	Movies
Daniel Radcliffe & Rupert Grint	6
Daniel Radcliffe & Emma Watson	5
Emma Watson & Rupert Grint	5
Triple	
Daniel Radcliffe, Emma Watson, Rupert Grint	5

Table 4: Frequent actor pairs and triple ($s = 5$).

6.2 Association Rules

The top association rules by confidence are:

Antecedent	Consequent	Confidence	Lift
{Rupert Grint}	{Daniel Radcliffe}	1.000	166.67
{Daniel Radcliffe}	{Rupert Grint}	1.000	166.67
{Emma W., R. Grint}	{Daniel Radcliffe}	1.000	166.67
{D. Radcliffe, E. Watson}	{Rupert Grint}	1.000	166.67
{Rupert Grint}	{D. Radcliffe, E. Watson}	0.833	166.67
{Daniel Radcliffe}	{Emma Watson}	0.833	119.05
{D. Radcliffe, R. Grint}	{Emma Watson}	0.833	119.05
{Rupert Grint}	{Emma Watson}	0.833	119.05
{Daniel Radcliffe}	{Emma W., R. Grint}	0.833	166.67
{Emma Watson}	{Daniel Radcliffe}	0.714	119.05

Table 5: Top 10 association rules by confidence.

6.3 Algorithm Verification

Both PCY and SON find exactly the same frequent pairs as A-Priori:

	Frequent pairs	Identical to A-Priori
A-Priori	3	—
PCY	3	✓
SON	3	✓

Table 6: Verification that all three algorithms find the same frequent pairs.

7 Conclusion

Frequent itemsets. The most frequent actors in the top 1000 IMDB movies are mostly well-known Hollywood names: Robert De Niro, Tom Hanks, Al Pacino. Not a huge surprise given the dataset. The more interesting result is in the pairs and triples: Daniel Radcliffe, Emma Watson, and Rupert Grint dominate, appearing together in 5–6 movies. That's entirely driven by the Harry Potter franchise, which makes sense but also means the "association" here is really just co-casting in a series.

Association rules. The highest-confidence rules are things like `{Rupert Grint}` $\rightarrow$ `{Daniel Radcliffe}` and vice versa, both at confidence 1.0. That means every time Grint appears in the dataset, Radcliffe is there too. The lift of 166.67 confirms this isn't random: Radcliffe is 166 times more likely to appear alongside Grint than you'd expect by chance. In practice, these rules don't tell us much beyond "they were in the same franchise," but they do validate that the algorithm is picking up real patterns.

A-Priori vs PCY. PCY was faster than A-Priori at every dataset size tested: 0.003s vs 0.014s at 200 baskets, 0.011s vs 0.033s at 1000. The difference comes from Pass 1: PCY uses the extra memory to hash pairs into buckets and can eliminate candidates before

even counting them in Pass 2. On a dataset this small the gap is modest, but it would grow at larger scales where the number of candidate pairs explodes.

SON. SON splits the data into chunks, mines frequent itemsets locally with a reduced support threshold, then verifies the union of candidates on the full dataset. It found the same pairs and triple as A-Priori (Sets identical: True), which is expected: SON is designed to have no false negatives. The runtime was noticeably higher though (0.174s at 1000 baskets vs 0.011s for PCY), which makes sense: you're running multiple local passes plus a full global counting pass. At 1000 movies that overhead isn't worth it, but the two-pass structure maps to MapReduce, so the payoff would come at much larger scales with parallel chunk processing.

Scalability. PCY scales the most smoothly across all fractions tested. SON's growth is steeper, mostly due to the local+global pass overhead mentioned above. A-Priori sits in the middle. The current dataset is too small to really stress-test these differences: the rankings might shift with a dataset 10–100x larger.

References

[1] Rajaraman, A., Leskovec, J., and Ullman, J. D. (2014). *Mining of Massive Datasets* (2nd ed.). Cambridge University Press.

[2] Shankhdhar, H. (2020). *IMDB Dataset of Top 1000 Movies and TV Shows*. Kaggle.

[3] Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software*, 3(24), 638.

Declaration: *I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offenses in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.*